

```

DROP DATABASE IF EXISTS sailors;
CREATE DATABASE sailors;
USE sailors;

-- Create tables
CREATE TABLE IF NOT EXISTS Sailors (
    sid INT PRIMARY KEY,
    sname VARCHAR(35) NOT NULL,
    rating FLOAT NOT NULL,
    age INT NOT NULL
);

CREATE TABLE IF NOT EXISTS Boat (
    bid INT PRIMARY KEY,
    bname VARCHAR(35) NOT NULL,
    color VARCHAR(25) NOT NULL
);

CREATE TABLE IF NOT EXISTS Reserves (
    sid INT NOT NULL,
    bid INT NOT NULL,
    sdate DATE NOT NULL,
    FOREIGN KEY (sid) REFERENCES Sailors(sid) ON DELETE CASCADE,
    FOREIGN KEY (bid) REFERENCES Boat(bid) ON DELETE CASCADE
);

-- Insert data
INSERT INTO Sailors VALUES
(1, 'Albert', 5.0, 40),
(2, 'Nakul', 5.0, 49),
(3, 'Darshan', 9, 18),
(4, 'Astorm Gowda', 2, 68),
(5, 'Armstormin', 7, 19);

INSERT INTO Boat VALUES
(1, 'Boat_1', 'Green'),
(2, 'Boat_2', 'Red'),
(103, 'Boat_3', 'Blue');

INSERT INTO Reserves VALUES
(1, 103, '2023-01-01'),
(1, 2, '2023-02-01'),
(2, 1, '2023-02-05'),
(3, 2, '2023-03-06'),
(5, 103, '2023-03-06'),
(1, 1, '2023-03-06');

-- Display tables
SELECT * FROM Sailors;
SELECT * FROM Boat;
SELECT * FROM Reserves;

-- Queries
-- Find the colors of the boats reserved by Albert
SELECT DISTINCT b.color
FROM Boat b
JOIN Reserves r ON b.bid = r.bid
JOIN Sailors s ON r.sid = s.sid
WHERE s.sname = 'Albert';

-- OR
-- SELECT B.color
-- FROM BOAT B, SAILORS S, RSERVERS R
-- WHERE S.sname = 'Albert'

```

```

-- AND S.sid = R.sid
-- AND B.bid = R.bid;

-- Find all sailor IDs who have rating at least 8 or reserved boat 103
SELECT DISTINCT s.sid
FROM Sailors s
LEFT JOIN Reserves r ON s.sid = r.sid
WHERE s.rating >= 8 OR r.bid = 103;

-- OR
(SELECT sid FROM Sailors WHERE rating >= 8)
UNION
(SELECT sid FROM Reserves WHERE bid = 103);

-- Find the names of sailors who have not reserved a boat whose name contains
the string "storm". Order the names in ascending order.

SELECT DISTINCT s.sname
FROM Sailors s
LEFT JOIN Reserves r ON s.sid = r.sid
WHERE r.sid IS NULL
AND s.sname LIKE '%storm%'
ORDER BY s.sname ASC;

-- Find names of sailors who have reserved all boats
SELECT sname FROM Sailors s WHERE NOT EXISTS (
    SELECT * FROM Boat b WHERE NOT EXISTS (
        SELECT * FROM Reserves r WHERE r.sid = s.sid AND b.bid = r.bid
    )
);

-- Find the name and age of the oldest sailor
SELECT sname, age FROM Sailors WHERE age = (SELECT MAX(age) FROM Sailors);

-- For each boat reserved by at least 2 sailors aged 40+, find bid and avg age

SELECT r.bid, AVG(s.age) AS avg_age
FROM Reserves r
JOIN Sailors s ON r.sid = s.sid
WHERE s.age >= 40
GROUP BY r.bid
HAVING COUNT(DISTINCT s.sid) >= 5;

-- Views
-- Sailors sorted by rating in descending order
CREATE VIEW NamesAndRating AS
SELECT sname, rating FROM Sailors ORDER BY rating DESC;
SELECT * FROM NamesAndRating;

-- Sailors who reserved a boat on a given date
CREATE VIEW SailorsWithReservation AS
SELECT sname FROM Sailors s JOIN Reserves r ON s.sid = r.sid WHERE r.sdate =
'2023-03-06';
SELECT * FROM SailorsWithReservation;

-- Names and colors of boats reserved by sailors with rating 5

CREATE VIEW Boats_By_Rating AS
SELECT DISTINCT b.bname, b.color
FROM Boat b
JOIN Reserves r ON b.bid = r.bid
JOIN Sailors s ON r.sid = s.sid
WHERE s.rating = 5.0;

```

```

SELECT * FROM Boats_By_Rating;

-- Triggers
-- Prevent boat deletion if it has active reservations
DELIMITER //
CREATE TRIGGER CheckAndDelete
BEFORE DELETE ON Boat
FOR EACH ROW
BEGIN
    IF EXISTS (SELECT * FROM Reserves WHERE bid = OLD.bid) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Boat is reserved and cannot be deleted';
    END IF;
END;//
DELIMITER ;

DELETE FROM Boat WHERE bid = 103; -- Should fail due to active reservations

-- Prevent sailors with rating <3 from reserving a boat
DELIMITER //
CREATE TRIGGER BlockReservation
BEFORE INSERT ON Reserves
FOR EACH ROW
BEGIN
    IF EXISTS (SELECT * FROM Sailors WHERE sid = NEW.sid AND rating < 3) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sailor rating less than 3';
    END IF;
END;
//
DELIMITER ;

INSERT INTO Reserves VALUES (4, 2, '2023-10-01'); -- Should fail

-- Trigger to delete expired reservations
CREATE TABLE TempTable (
    last_deleted_date DATE PRIMARY KEY
);

DELIMITER //
CREATE TRIGGER DeleteExpiredReservations
BEFORE INSERT ON TempTable
FOR EACH ROW
BEGIN
    DELETE FROM Reserves WHERE sdate < CURDATE();
END;
//
DELIMITER ;

SELECT * FROM Reserves; -- Before deletion
INSERT INTO TempTable VALUES (CURDATE()); -- Deletes expired reservations
SELECT * FROM Reserves; -- After deletion

```